RISC240 Reference Manual

Leo Serodio
Original RISC240 ISA and manual by Bill Nace & Saugata Ghose
Version 2.0.0
17 July 2026

1. Introduction

RISC240 is a special-purpose instruction set architecture (ISA) designed for teaching the structure and design of digital systems. The RISC240 ISA consists of word size and addressing definitions (Section 3), the visible state (Section 4), and assembly instructions (Section 5). You can use the assembly instructions and assembler pseudo-operations (Section 6) to write assembly programs (Section 7). The as240 assembler (Section 8) and the sim240 simulator (Section 9) are used to assemble and test assembly programs written using the RISC240 instructions, and synthesizable SystemVerilog is provided for a working RISC240 CPU (Section 10). Section 11 documents the RISC240-ML extension, a small vector execution subsystem for quantized machine-learning workloads, including the ACCST instruction added in this revision, along with the mlasm assembler and mlsim simulator used to assemble and debug RISC240-ML programs.

RISC240 is based upon the RISC-V ISA (https://riscv.org/), but has been significantly modified and simplified for educational purposes. As a result, a number of key components were not included in the design of RISC240, to ease the learning curve for introductory students. RISC240 is not intended to be a high-performance ISA, so don't expect superscalar features, pipelining, multi-threading, etc. Those features — and how a real-world ISA such as RISC-V enables them — are typically covered in a follow-on computer architecture course.

2. Overview on ISAs

An instruction set architecture is an interface specification – a set of guidelines – about how to program a CPU or family of CPUs. Sometimes we say something like that “an ISA is a contract between the chip designer (i.e., the computer architect) and the programmers.” What this means is that both sides (the programmer and the chip designer) need to have a common vision about how to get the CPU to run programs properly. The chip designers (typically a team of designers) have built the CPU to work in a certain way, and they need to be able to tell programmers these details. However, the chip designers don't just want to give programmers a circuit diagram or a datapath picture — most programmers aren't computer architects, so those communication methods don't work well. Instead, the ISA is a list of instructions and other details about how the chip works, without exposing the complete specification of how the chip works.

If the complete specifications of a chip were exposed, programmers may write software that depends on specific details about this chip. This would prevent designers from changing those specifications on future chips, which could severely limit improvements to chip performance. By exposing only enough information about the interface between programs and the chip, chip designers can build new chips with significant under-the-hood changes, and programmers can run their existing programs on these new chips without having to rewrite or recompile their code.

3. RISC240 Word Sizes and Addressing

The RISC240 ISA makes use of 16-bit words. In other words, all operations are done on 16 bits of data at a time. Recall that a byte consists of 8 bits, which means that one word in RISC240 is made up of two bytes.

RISC240 assumes that all addresses are byte-addressable. This means that each byte in memory has its own address. Because everything in the RISC240 operates at the granularity of a word and not a byte, we simplify the ISA by providing automatic word alignment: the least-significant bit of an address is always replaced by a zero when accessing memory (and in special registers such as the program counter; see Section 4), ensuring that no operation takes place on multiple words at once. For example, if you try to branch to address 0x0053, a RISC240 CPU will automatically set the program counter to 0x0052.

4. Visible State in RISC240

The assembly language programmer selects instructions and orders them such that they have a particular effect on a RISC240 CPU. Through this process, the programmer is manipulating the visible state of the CPU. The RISC240 ISA provides the following pieces of visible state:

• Registers (r0 – r7): Eight general-purpose registers, each 16 bits wide. The registers are named r0 through r7. Register r0 is known as the zero register – it is hardwired to the value zero, and any writes to r0 are ignored by the CPU.

• Program Counter (PC): A 16-bit wide register that, at the beginning of the first clock cycle of the execution of an instruction, holds the memory address of that instruction.

• Condition Code Flags (CC or ZCNV): Four single-bit flags that are set based on the outcomes of an ALU operation. We usually list them in the order ZCNV, where each of the letters represents one of the following flags:

– Zero Flag (Z): This bit is set (i.e., Z=1) if the result of the ALU operation was zero.

– Carry Flag (C): This bit is the carry-out bit of the ALU operation.

– Negative Flag (N): This bit is the most significant bit of the result of the ALU operation. Therefore, if it is set (i.e., N=1), that means the result was a negative value.

– Overflow Flag (V): This bit is set if the ALU operation resulted in overflow.

• Memory: RISC240 includes a specification of memory, unlike many CPUs. In our case, the memory is an array of 65,536 (i.e., 2^16) bytes, where reads and writes are done on 16-bit words. As such, it has a 16-bit address and a 16-bit data bus. M[8] represents the 16-bit word stored in byte addresses 8 and 9. The memory has combinational read and sequential write capabilities.

• Vector Registers (v0 – v7): Eight 64-bit vector registers added by the RISC240-ML extension. Unlike scalar register r0, vector register v0 is not hardwired to zero. See Section 11 for details.

• Accumulator (ACC): A signed 32-bit accumulator added by the RISC240-ML extension, used by the VDOT and ACCST instructions and clearable with VACLR. See Section 11 for details.

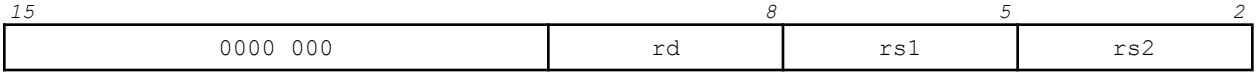
5. RISC240 Assembly Instructions

ADD rd, rs1, rs2
Addition

Semantics:
`rd ← rs1 + rs2`

CC Flags:
ZCNV - set normally

Encoding:



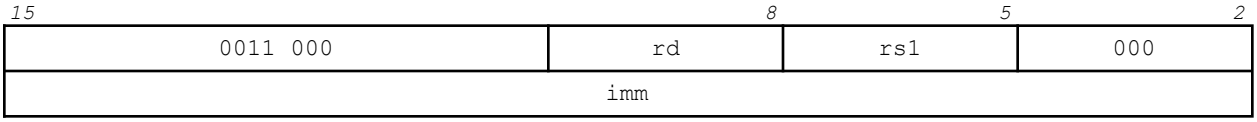
Cycles:
5

ADDI rd, rs1, imm
Add Immediate

Semantics:
rd ← rs1 + imm

CC Flags:
ZCNV - set normally

Encoding:



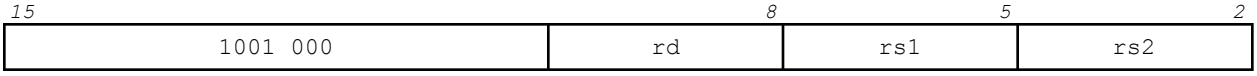
Cycles:
7

AND rd, rs1, rs2
Bitwise AND

Semantics:
rd ← rs1 · rs2

CC Flags:
ZN - set normally
C = 0
V = 0

Encoding:



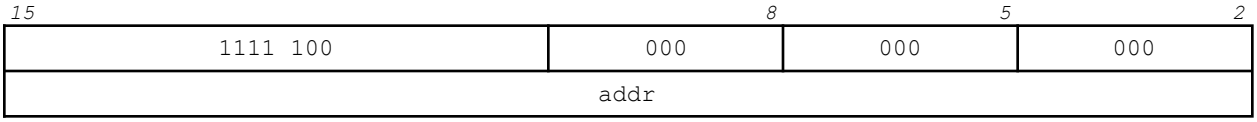
Cycles:
5

BRA addr
Branch Always

Semantics:
`PC ← addr`

CC Flags:
`not changed`

Encoding:



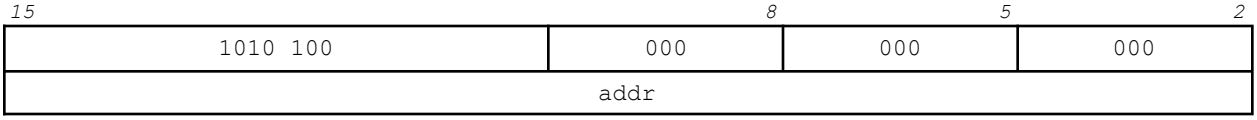
Cycles:
7

BRC addr
Branch If Carry

Semantics:
`if(C) PC ← addr`

CC Flags:
`not changed`

Encoding:



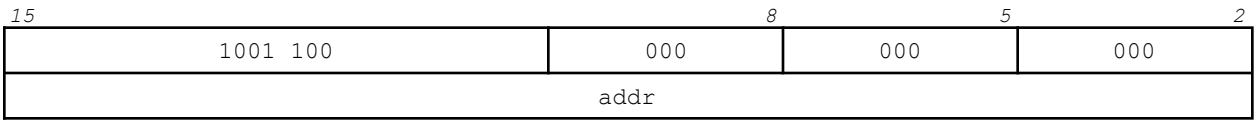
Cycles:
`7 if taken; 6 if not taken`

BRN addr
Branch If Negative

Semantics:
`if(N) PC ← addr`

CC Flags:
`not changed`

Encoding:



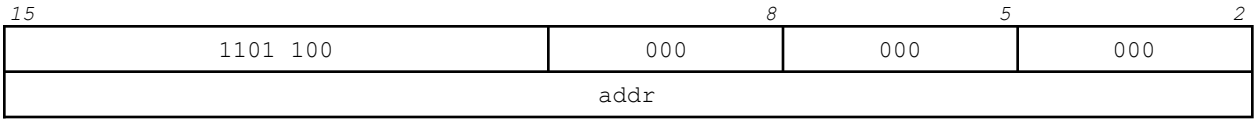
Cycles:
`7 if taken; 6 if not taken`

BRNZ addr
Branch If Negative or Zero

Semantics:
`if(N OR Z) PC ← addr`

CC Flags:
`not changed`

Encoding:



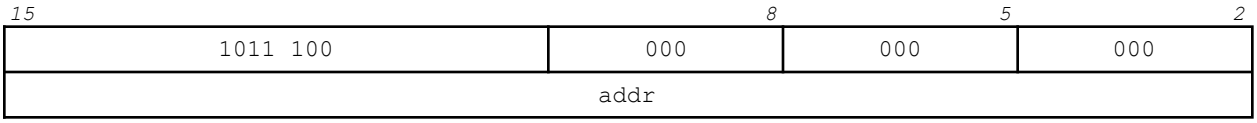
Cycles:
`7 if taken; 6 if not taken`

BRV addr
Branch If Overflow

Semantics:
`if(V) PC ← addr`

CC Flags:
`not changed`

Encoding:



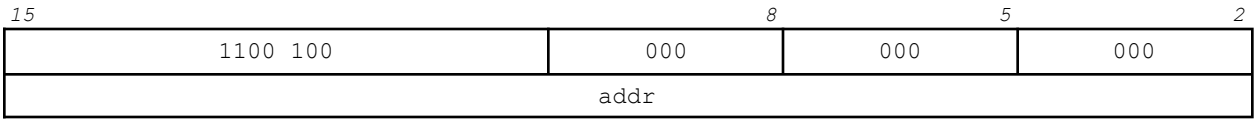
Cycles:
`7 if taken; 6 if not taken`

BRZ addr
Branch If Zero

Semantics:
`if(Z) PC ← addr`

CC Flags:
`not changed`

Encoding:



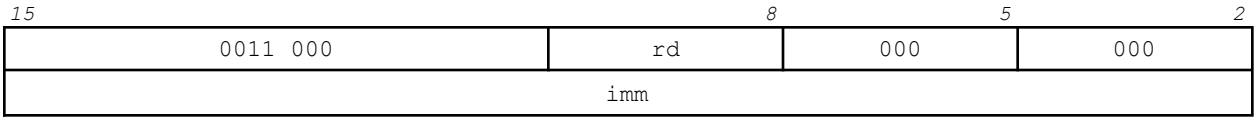
Cycles:
`7 if taken; 6 if not taken`

LI rd, imm
Load Immediate

Semantics:
`rd ← imm`

CC Flags:
`ZCNV - set normally`

Encoding:



Cycles:
`7`

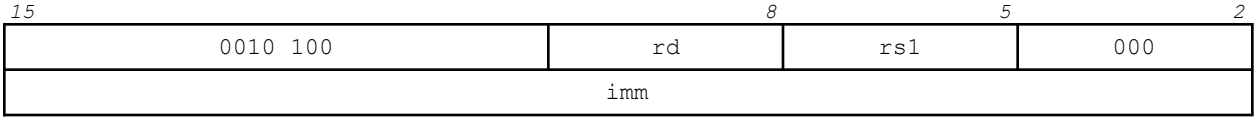
Notes:
Implemented in hardware as ADDI rd, r0, imm.

LW rd, rs1, imm
Load Word

Semantics:
`rd ← M[rs1 + imm]`

CC Flags:
`ZN - set normally`
`C = 0`
`V = 0`

Encoding:



Cycles:
`9`

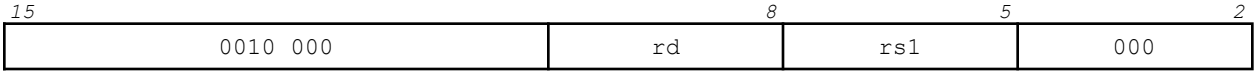
MV rd, rs1
Move Register

Semantics:

rd ← rs1

CC Flags:
not changed

Encoding:



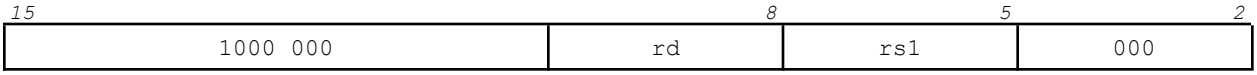
Cycles:
5

NOT rd, rs1
Bitwise NOT

Semantics:
rd ← NOT rs1

CC Flags:
ZN - set normally
C = 0
V = 0

Encoding:



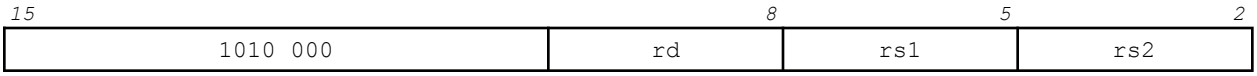
Cycles:
5

OR rd, rs1, rs2
Bitwise OR

Semantics:
rd ← rs1 OR rs2

CC Flags:
ZN - set normally
C = 0
V = 0

Encoding:



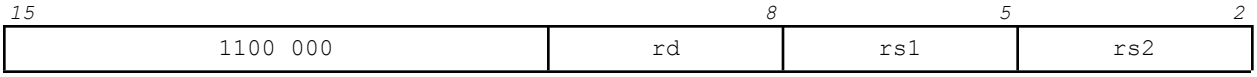
Cycles:
5

SLL rd, rs1, rs2
Shift Left Logical

Semantics:
```
rd ← rs1 << rs2[3:0]
```

CC Flags:
```
ZN - set normally
C = 0
V = 0
```

Encoding:



Cycles:
5

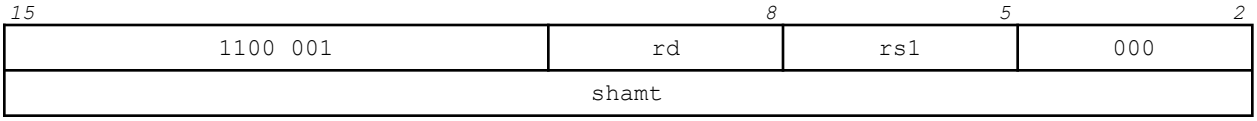
Notes:
For a shift amount of n, shifts in n zeros from the right.

SLLI rd, rs1, shamt
Shift Left Logical Immediate

Semantics:
```
rd ← rs1 << shamt[3:0]
```

CC Flags:
```
ZN - set normally
C = 0
V = 0
```

Encoding:



Cycles:
7

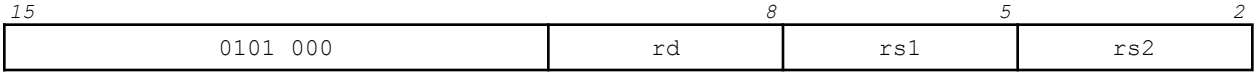
Notes:
For a shift amount of n, shifts in n zeros from the right.

SLT rd, rs1, rs2
Set If Less Than

Semantics:
```
if(rs1 < rs2) rd ← 16'b1
else rd ← 16'b0
```

CC Flags:
```
ZCNV - set based on result of (rs1 - rs2)
```

Encoding:



Cycles:
```
6
```

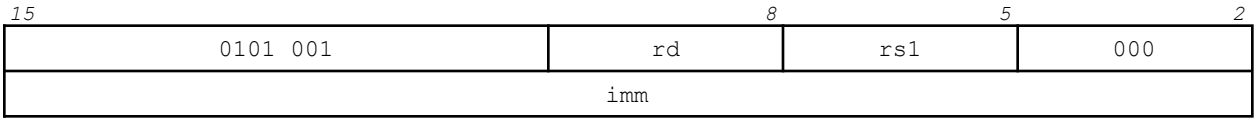
Notes:
Two's complement comparison.

SLTI rd, rs1, imm
Set If Less Than Immediate

Semantics:
```
if(rs1 < imm) rd ← 16'b1
else rd ← 16'b0
```

CC Flags:
```
ZCNV - set based on result of (rs1 - imm)
```

Encoding:



Cycles:
```
8
```

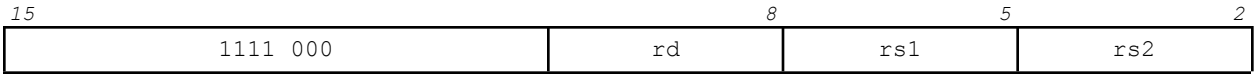
Notes:
Two's complement comparison.

SRA rd, rs1, rs2
Shift Right Arithmetic

Semantics:
```
rd ← rs1 >>> rs2[3:0]
```

CC Flags:
```
ZN - set normally
C = 0
V = 0
```

Encoding:



Cycles:
5

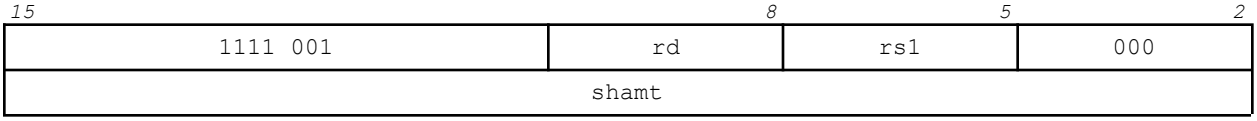
Notes:
For a shift amount of n, shifts in n copies of rs1[15] from the left.

SRAI rd, rs1, shamt
Shift Right Arithmetic Immediate

Semantics:
```
rd ← rs1 >>> shamt[3:0]
```

CC Flags:
```
ZN - set normally
C = 0
V = 0
```

Encoding:



Cycles:
7

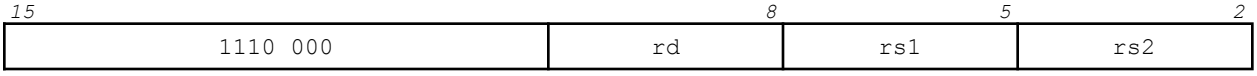
Notes:
For a shift amount of n, shifts in n copies of rs1[15] from the left.

SRL rd, rs1, rs2
Shift Right Logical

Semantics:
```
rd ← rs1 >> rs2[3:0]
```

CC Flags:
```
ZN - set normally
C = 0
V = 0
```

Encoding:



Cycles:
5

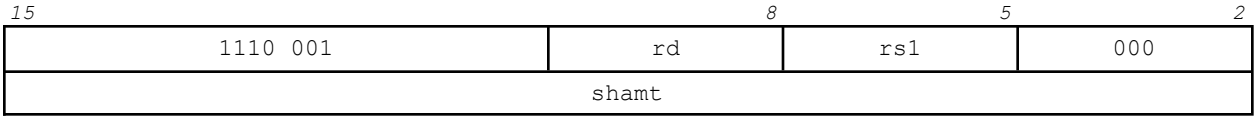
Notes:
For a shift amount of n, shifts in n zeros from the left.

SRLI rd, rs1, shamt
Shift Right Logical Immediate

Semantics:
```
rd ← rs1 >> shamt[3:0]
```

CC Flags:
```
ZN - set normally
C = 0
V = 0
```

Encoding:



Cycles:
```
7
```

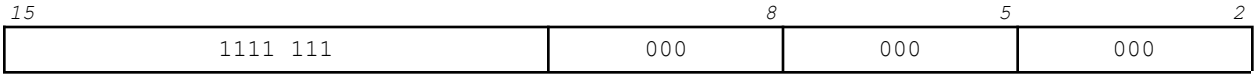
Notes:
For a shift amount of n, shifts in n zeros from the left.

STOP
Stop Execution

Semantics:
```
PC ← PC
```

CC Flags:
```
not changed
```

Encoding:



Cycles:
```
5
```

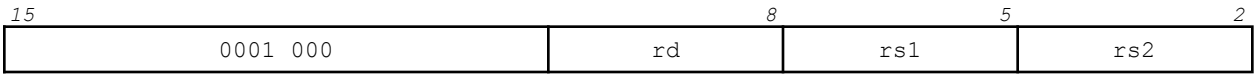
Notes:
Used to halt the CPU when program execution is finished, by looping infinitely on the STOP instruction until a reset signal is asserted.

SUB rd, rs1, rs2
Subtraction

Semantics:
```
rd ← rs1 - rs2
```

CC Flags:
```
ZCNV - set normally
```

Encoding:



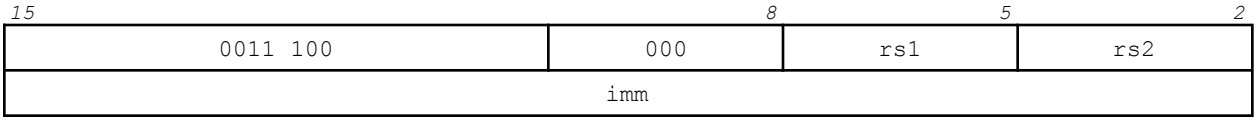
Cycles:
5

SW rs1, rs2, imm
Store Word

Semantics:
M[rs1 + imm] ← rs2

CC Flags:
ZN - set normally
C = 0
V = 0

Encoding:



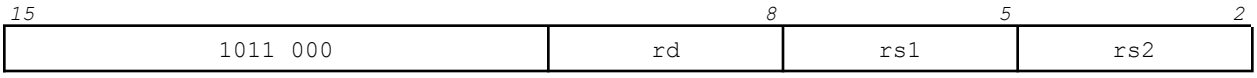
Cycles:
9

XOR rd, rs1, rs2
Bitwise XOR

Semantics:
rd ← rs1 ⊕ rs2

CC Flags:
ZN - set normally
C = 0
V = 0

Encoding:



Cycles:
5

---

6. Assembler Pseudo-Operations

Pseudo-operations are directives that show up in an assembly program, but are intended to communicate with the assembler as it prepares the machine language. Pseudo-operations don't communicate anything to the CPU (unlike an instruction), and so are not part of the ISA. Unlike an instruction, there is no machine language equivalent for each pseudo-operation: they don't show up in the machine language file, though they do affect how the assembler shapes the machine language. The as240 assembler (which assembles RISC240 assembly programs into machine language) provides three pseudo-operations (all of which start with a period):

.ORG addr — Origin
This pseudo-operation tells the assembler that the following instruction should be placed at address addr. Normally, when .ORG is not used, the next line will have an address that is zero, two, or four more than the current line (depending on if this line is blank, has a short instruction on it, or has a long instruction on it). The .ORG pseudo-operation interrupts this incremental processing and allows you to put data or instructions at any place in the RISC240 memory map.

.DW value — Data Word
This pseudo-operation tells the assembler that you want a particular value to be placed in memory at the address of this line. The value can be specified as a hexadecimal value or as a label.

.EQU value — Equals
This pseudo-operation assigns a constant value to a particular label. The label on this line does not equate to an address, unlike all other labels. Rather, the compiler finds all instances where the label is used as an operand, and replaces them with the constant value when converting assembly into machine code.

7. Assembly Program Format

An assembly program is written as a text file, in ASCII format, much like other programming languages. The file is named with a .asm extension. Assembly programs are line-oriented, which means that the line stands alone. All information about a single instruction is contained in a single line. There is no way to have an instruction span multiple lines. On the other hand, not all lines have an instruction on them — they may be blank, contain a comment or a pseudo-operation. The entire assembly file is case-insensitive. You may use lower or upper case characters, but the assembler looks at the file as if all characters had been converted to upper case.

On any line of the assembly file, up to four fields will occur, in the following order. Any line containing a label or operand must also have an instruction. Each field is separated by some form of whitespace (spaces, tabs, etc.).

1. Label
The label specifies a human-usable name for the address of the instruction on that line. Labels also can be used to name a constant specified in an .EQU pseudo-operation. The label is an alphanumeric value starting at the beginning of the line. There should be no whitespace before the label. Labels are case insensitive (as is the entire file). Labels may be any length. Each label must be unique — there can be no other label in the file with the same value. You may use any combination of letters, digits, and the underscore ('_') character for a label, though a meaningful name is strongly recommended. Labels that look like register names (r4) or instruction mnemonics (LDA) are legal but discouraged.

2. Instruction
This is the mnemonic for the instruction. It must match one of the given instructions in the RISC240 ISA (Section 5), one of the RISC240-ML vector instructions (Section 11), or one of the assembler pseudo-operations (Section 6).

3. Operands
Depending upon the instruction, there should be 0–3 operands. If there are two or more operands, they must be separated by a comma, and may have whitespace before or after the comma. The nature of the operand is determined by the instruction. Operands can be register names, immediate values, memory addresses, or labels. Register names start with the 'r' character, and are followed by a single digit ranging between 0 and 7, inclusive; vector register names start with the 'v' character and are followed by a single digit ranging between 0 and 7, inclusive. Immediate values and memory addresses can be numerical values or label names. Numerical values are hexadecimal numbers of no more than 16 bits (i.e., 0 through FFFF). All hexadecimal values must start with the dollar sign ('$'). Leading zeros are not necessary. Label names must match a value in the label field somewhere in your assembly code.

4. Comment
A comment is specified by using the semicolon (';'). All text on the line following the semicolon is ignored by the assembler.

If a field does not exist (like a label), the separation whitespace must still exist. Therefore, a label is the only thing that can be at the beginning of a line. A line without a label needs at least one space before the instruction.

8. RISC240 Assembler: as240

as240 is an assembler for the RISC240 ISA, provided as part of your course toolchain. It is a Python 3 program and requires a Python 3.5+ installation to execute. When you run the assembler, it takes the following arguments:

```
> as240 [options] <base_name>.asm
```

When specifying the file to assemble (<base_name>.asm), as240 assumes a default extension of .asm, but this can be overridden by specifying the full filename. as240 produces three files once it finishes running:

• List File: Shows your code and how it was converted by the assembler. All of the memory values for all of the instructions are shown, as well as label values, data words, etc. By default, the .list file has the same base name as the .asm file (e.g., assembling lab4.asm produces lab4.list).

• Simulation Memory Image File: Shows all of the addresses and the values that should be stored in each address. By default, the file is called memory.hex.

• Synthesis Memory Image File: Like the memory.hex file, shows addresses and values that should be stored in each address. This is a different format file, used by the memory model for synthesis. This file is called memory.mif. At the current time, there is no way to use a different output filename for this file.

The following command-line options are available for as240:

```
-h
--help          provides short help text and usage information, and exits
-m <filename>
--mfile=<filename>       uses the specified filename for the memory image file
-l <filename>
--listfile=<filename>    uses the specified filename for the list file
-s <filename>
--symbolfile=<filename>  outputs a symbol list at the specified filename
-o              sends the list file output to stdout (for piping into sim240)
                instead of a file, and writes no files to the directory
--version       prints the current version number, and exits
```

9. RISC240 Simulator: sim240

sim240, the RISC240 simulator, is a great resource to understand (micro-)instruction operations and debug programs. Like as240, sim240 is a perl program provided as part of your course toolchain.

When you run the simulator, it takes the list file generated by as240 (e.g., lab4.list) as an argument:

```
> sim240 [options] <base_name>.list
```

The following command-line options are available for sim240:

```
-h
--help          provides short help text and usage information, and exits
-r
--run           run only: doesn't accept user input, but rather starts the
                program, and outputs all simulation steps until a STOP
                instruction is reached (or some large number of clock
                cycles has passed)
-n
--norandom      initializes each word in memory to zeros, instead of to
                random values
-t <filename>
--transcript=<filename>  outputs a transcript of the simulator at the
                specified filename
-q
--quiet         doesn't print output with step/ustep
-g <filename>
--grade=<filename>       runs simulation, checks state against the
                specified file, and exits
-i              reads the list file from stdin instead of from a list
                file (this lets you pipe your as240 output to sim240
                without generating .list/.hex files)
-v
--version       prints the current version number, and exits
```

9.1. sim240 Commands

Unlike as240, sim240 is an interactive program that allows the user to specify what should happen throughout its use. When the simulator is invoked, a prompt will be printed on the terminal and the simulator will await user input. At any prompt, the user can enter any of the following commands:

```
quit / q / exit    quits the simulator
help / h            prints a help screen
reset               resets the processor state to where it was when the
                    .list file was loaded: memory and registers
                    (including PC) return to initial values, and all
                    breakpoints are cleared
run [n] / r [n]     simulates the next n instructions; if n is not
                    specified, simulate until a STOP instruction is
                    encountered; if a breakpoint is encountered, it
                    will halt the run
step / s            simulates a single instruction; technically,
                    simulates until the next FETCH state is
                    encountered
ustep / u           simulates a single micro-instruction; that is, a
                    single state or single clock
break [addr/label]  sets a breakpoint
lsbrk               lists all the breakpoints that you have previously
                    set
clear [addr/label]  clears a breakpoint that you have previously set
                    for this address
save <filename>     saves the current state of the simulation to a
```

```
                    file; includes your current location in the
                    program, all memory contents, and all breakpoints
load <filename>     loads a simulation file that was created earlier
                    using the save command
```

9.2. Querying RISC240 State

You may ask about the contents of memory or a register by using the question mark command. Simply specify a register (including microarchitectural registers like MAR and IR) and then follow it with a question mark:

```
> r3?
> MAR?
> IR?
```

If you want to see the entire register file, then you can use the wildcard asterisk:

```
> r*?
r0: 0000 r1: 1492
r2: 0410 r3: 1234
r4: BEEF r5: 0000
r6: 8787 r7: A492
```

You can also view all microarchitectural registers (e.g., MAR, IR) with the *? command, though the output isn't quite as pretty:

```
> *?
0208 FETCH 283E 0000 0000 283D 0000 0000 1492 0410 1234 BEEF 0000 8787 A492
```

Similarly, you can ask about a memory location. Use the m array notation with an address between square brackets. You may use m or mem (or their capitalized versions). You need not specify leading zeros. You must use an address in hexadecimal format, and not a label. By using a colon to delimit start and end addresses, you can ask about a range of values. Recall that memory is byte-addressable (see Section 3), but sim240 will always print the entire word in a word-aligned manner automatically.

```
> m[0000]?
> M[1234]?
> mem[20:87]?
```

For each memory address, you will be told the value that the word at that location holds, as well as what that word decodes to if it were an instruction. For example:

```
> m[7:a]
mem[0006:0007]: 31C0 ADDI 7 0 0
mem[0008:0009]: 0001 ADD 0 0 1
mem[000A:000B]: F0F0 SRA 3 2 0
```

Note that this example is asking about two instructions: ADDI and SRA. ADDI is a two-word instruction, but the printing routine doesn't try to disassemble the values stored in memory. As a result, the value stored in mem[0008], $0001, is printed as an ADD instruction, even though it is supposed to be an immediate value, because that's what the first word of an ADD instruction looks like. Just be a bit careful about interpreting what you see.

9.3. Setting RISC240 State

You can use similar means to change the state of a register or memory. In this case, instead of ending with a question mark, use an equals sign:

```
> r2=d2
> MAR=0100
> mem[290]=7734
```

Again, when you change the value in a memory location, sim240 will automatically word-align it. As a result, mem[291]=7734 has the same exact effect as mem[290]=7734. You cannot currently set a range of memory values with one command.

10. RISC240 Synthesizable Processor

A RISC240 processor has been described in synthesizable SystemVerilog, which can be programmed onto an Altera FPGA board. The processor has an FSM which controls the datapath. The datapath contains a variety of components, including an arithmetic logic unit, a register file, and multiplexers. The source is organized into the following files:

```
File            Contents
alu.sv          the arithmetic logic unit for the RISC240 processor
bram*           three files used to instantiate a block RAM (BRAM)
                for the program memory; memory.mif initializes the
                values in the memory
constants.sv    definitions of readable names for bit patterns like
                ALU operations, microcode operations, control points,
                etc.
controlpath.sv  the FSM (i.e., control path) that drives the datapath
datapath.sv     a collection module where all the components in the
                datapath are instantiated
library.sv      a collection of parameterized components
regfile.sv      the register file, with eight 16-bit registers, and
                with register 0 hardwired to the value zero
RISC240.sv      the top-level module, which instantiates the
                datapath, control path, and I/O modules, including an
                I/O module for simulation
```

These files allow synthesis, with a few caveats:

• At the top of constants.sv, there is a comment (`define synthesis) that needs to be uncommented before synthesizing. Make sure to re-comment it for any simulation tasks.

• These files have no method for getting input to your program, nor receiving output from it. Memory-mapped I/O ports need to be created and connected to LEDs and switches for interactive designs.

• Machine code needs to be included in the synthesis run. The as240 assembler outputs a memory image file (named memory.mif by default, not the memory.hex file). This file is a memory image of your program. If it is included in the project, your synthesis tool will find it during synthesis and use it to configure the memory.

• Program memory starts at address $0000 and ends at $03FF. Locations $0400 through $07FF are for data (variables).

11. RISC240-ML ISA Extension

11.1. Overview

RISC240-ML extends the base scalar RISC240 instruction set with a small vector execution subsystem intended for quantized machine-learning workloads.

The base scalar RISC240 ISA remains unchanged. This section only specifies the additional architectural state and instructions introduced by the RISC240-ML extension. Sections 3–10 above should be used for the scalar instruction set, addressing rules, condition codes, and general assembly-language format.

11.2. Added Architectural State

Vector Registers
RISC240-ML adds eight 64-bit vector registers:

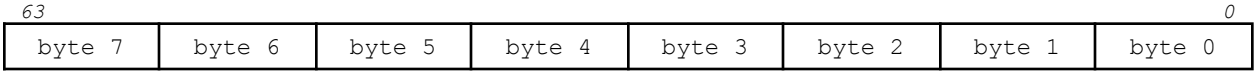
```
v0, v1, v2, v3, v4, v5, v6, v7
```

Unlike scalar register r0, vector register v0 is not hardwired to zero.

Each vector register may be interpreted in two ways:

• Memory-transfer view: four 16-bit words

• Arithmetic view: eight 8-bit elements



Vector arithmetic instructions operate independently on the eight 8-bit elements.

Vector load and store instructions transfer the same 64-bit value as four consecutive 16-bit memory words.

Accumulator
RISC240-ML adds one signed 32-bit accumulator:

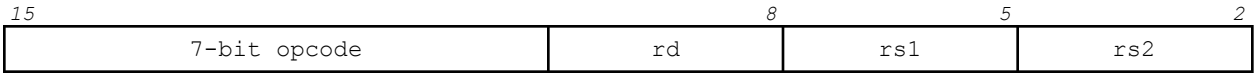
```
ACC
```

The accumulator is used by the VDOT and ACCST instructions, and may be cleared with VACLR.

11.3. General Instruction Encoding

All instruction words remain 16 bits wide.

The new vector instructions use the same basic register-field locations as the scalar RISC240 instructions:



For vector arithmetic:

• rd selects the destination vector register

• rs1 selects the first source vector register

• rs2 selects the second source vector register

For vector and accumulator memory instructions, scalar and vector register fields are used as described in the individual instruction definitions.

11.4. Opcode Summary

Instruction	7-bit opcode	Base instruction word
VADD	0110000	$6000
VMUL	0110001	$6200
VRELU	0110010	$6400
VDOT	0110011	$6600
VACLR	0110100	$6800
VLD	0110101	$6A00
ACCST	0100001	$4200
VST	0111011	$7600

The base instruction word assumes all register fields are zero.

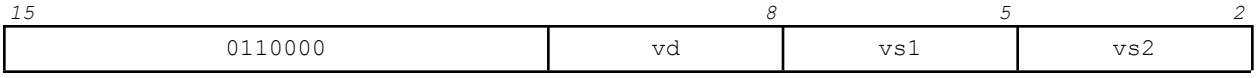
11.5. Vector Instructions

VADD vd, vs1, vs2
Vector Addition

Semantics:
```
for i = 0 to 7:
    vd.byte[i] <- (vs1.byte[i] + vs2.byte[i]) mod 256
```

CC Flags:
```
ZCNV: not changed
```

Encoding:



Cycles:
```
5
```

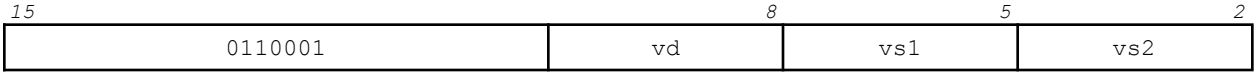
Notes:
Each 8-bit element is added independently. Carry out from one element does not propagate into another element.

VMUL vd, vs1, vs2
Vector Multiplication

Semantics:
```
for i = 0 to 7:
    vd.byte[i] <- low_8_bits(vs1.byte[i] * vs2.byte[i])
```

CC Flags:
ZCNV: not changed

Encoding:



Cycles:
5

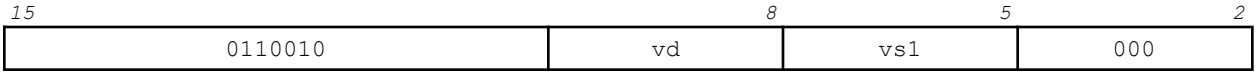
Notes:
Each pair of 8-bit elements is multiplied independently. The current implementation stores only the low eight bits of each product.

VRELU vd, vs1
Vector Rectified Linear Unit

Semantics:
```
for i = 0 to 7:
    if vs1.byte[i][7] == 1:
        vd.byte[i] <- 0
    else:
        vd.byte[i] <- vs1.byte[i]
```

CC Flags:
ZCNV: not changed

Encoding:



Cycles:
5

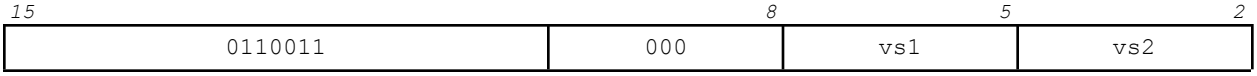
Notes:
Each 8-bit element is interpreted as a signed two's-complement value. Negative elements are replaced with zero; nonnegative elements pass through unchanged.

VDOT vs1, vs2
Signed Vector Dot Product and Accumulate

Semantics:
```
dot <- 0
for i = 0 to 7:
    dot <- dot + signed(vs1.byte[i]) * signed(vs2.byte[i])
ACC <- ACC + dot
```

CC Flags:
ZCNV: not changed

Encoding:



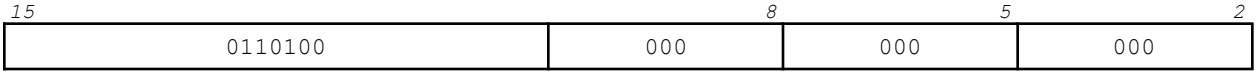
Cycles:
5

Notes:
Each vector is interpreted as eight signed 8-bit elements. The eight signed products are accumulated into a signed 32-bit dot-product result, which is then added to ACC.

VACLR
Clear Vector Accumulator

Semantics:
```
ACC <- 0
```

CC Flags:
```
ZCNV: not changed
```

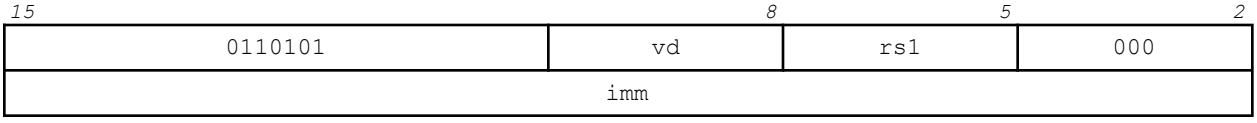
Encoding:


Cycles:
5

VLD vd, rs1, imm
Vector Load

Semantics:
```
EA <- rs1 + imm
vd[15:0]  <- M[EA]
vd[31:16] <- M[EA + 2]
vd[47:32] <- M[EA + 4]
vd[63:48] <- M[EA + 6]
```

CC Flags:
```
ZCNV: not changed
```

Encoding:


Cycles:
16

Notes:

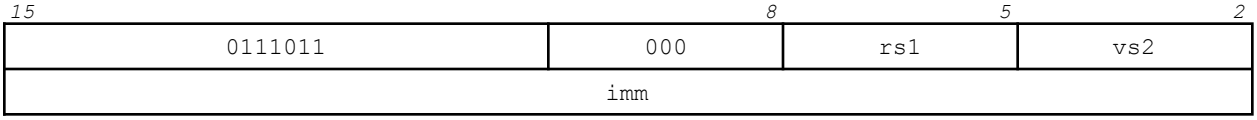
VLD loads one 64-bit vector from four consecutive 16-bit memory words. The effective address is word-aligned using the normal RISC240 addressing rules.

VST rs1, vs2, imm
Vector Store

Semantics:
```
EA <- rs1 + imm
M[EA]     <- vs2[15:0]
M[EA + 2] <- vs2[31:16]
M[EA + 4] <- vs2[47:32]
M[EA + 6] <- vs2[63:48]
```

CC Flags:
```
ZCNV: not changed
```

Encoding:



Cycles:
15

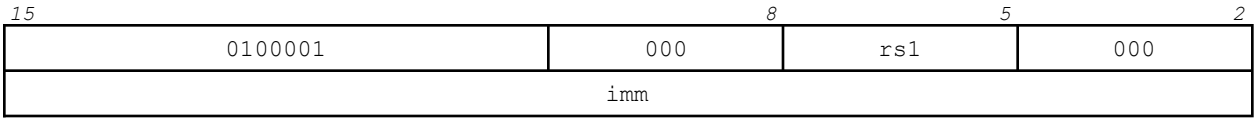
Notes:
VST stores one 64-bit vector into four consecutive 16-bit memory words. The effective address is word-aligned using the normal RISC240 addressing rules.

ACCST rs1, imm
Accumulator Store

Semantics:
```
EA <- rs1 + imm
M[EA]     <- ACC[15:0]
M[EA + 2] <- ACC[31:16]
```

CC Flags:
```
ZCNV: not changed
```

Encoding:



Cycles:
11

Notes:

ACCST stores the signed 32-bit accumulator to two consecutive 16-bit memory words, low half first. The effective address is word-aligned using the normal RISC240 addressing rules. ACCST does not modify ACC.

11.6. Assembly Examples

Vector Addition

```
        LI      r1, $0100

        VLD     v1, r1, 0
        VLD     v2, r1, 8

        VADD    v3, v1, v2

        VST     r1, v3, 16
        STOP
```

This program:

1. Loads a vector from $0100 into v1

2. Loads a vector from $0108 into v2

3. Adds the vectors element by element

4. Stores the result beginning at $0110

Dot Product and Accumulator Store

```
        LI      r1, $0100

        VLD     v1, r1, 0
        VLD     v2, r1, 8

        VACLR
        VDOT    v1, v2

        LI      r2, $0120
        ACCST   r2, 0
        STOP
```

After execution, ACC contains the signed dot product of v1 and v2, and the 32-bit accumulator value has also been written to memory beginning at $0120.

11.7. Memory Layout Example

The following data defines two 64-bit vectors beginning at $0100.

```
        .ORG    $0100

        ; Vector 1
        .DW     $0201
        .DW     $0403
        .DW     $0605
        .DW     $0807

        ; Vector 2
        .DW     $0101
        .DW     $0101
```

```
        .DW     $0101
        .DW     $0101
```

After loading the first vector:

```
v1 = $0807060504030201
```

The lowest-addressed memory word becomes bits [15:0] of the vector register.

11.8. RISC240-ML Assembler: mlasm

mlasm is the assembler for the RISC240-ML extension. It is a drop-in replacement for as240 that additionally recognizes the vector register names v0–v7, the accumulator ACC, and the RISC240-ML instructions (VADD, VMUL, VRELU, VDOT, VACLR, VLD, VST, ACCST) alongside the full base RISC240 instruction set. Programs that use only scalar instructions assemble identically under mlasm and as240.

When you run the assembler, it takes the following arguments:

```
> mlasm [options] <base_name>.asm
```

Like as240, mlasm produces a list file, a simulation memory image file (memory.hex), and a synthesis memory image file (memory.mif). The following command-line options are available for mlasm:

```
-h
--help          provides short help text and usage information, and exits
-m <filename>
--mfile=<filename>       uses the specified filename for the memory image file
-l <filename>
--listfile=<filename>    uses the specified filename for the list file
-s <filename>
--symbolfile=<filename>  outputs a symbol list at the specified filename
-o              sends the list file output to stdout (for piping into mlsim)
                instead of a file, and writes no files to the directory
--version       prints the current version number, and exits
```

mlasm reports the same category of syntax and label errors as as240, plus additional checks specific to the vector extension: an operand outside v0–v7 given where a vector register is expected, a scalar register given where a vector register is expected (or vice versa), and vector instructions used without the RISC240-ML opcode range enabled.

11.9. RISC240-ML Simulator: mlsim

mlsim is the simulator for the RISC240-ML extension. It is a drop-in replacement for sim240 that adds the vector register file (v0–v7) and the accumulator (ACC) to the visible simulation state, alongside all scalar registers and memory.

When you run the simulator, it takes the list file generated by mlasm (e.g., lab6.list) as an argument:

```
> mlsim [options] <base_name>.list
```

The following command-line options are available for mlsim:

```
-h
--help          provides short help text and usage information, and exits
-r
--run           run only: doesn't accept user input, but rather starts the
                program, and outputs all simulation steps until a STOP
                instruction is reached (or some large number of clock
                cycles has passed)
-n
```

```
--norandom      initializes each word in memory to zeros, instead of to
                random values
-t <filename>
--transcript=<filename>  outputs a transcript of the simulator at the
                specified filename
-q
--quiet         doesn't print output with step/ustep
-g <filename>
--grade=<filename>       runs simulation, checks state against the
                specified file, and exits
-i              reads the list file from stdin instead of from a list
                file (this lets you pipe your mlasm output to mlsim
                without generating .list/.hex files)
-v
--version       prints the current version number, and exits
```

mlsim accepts every sim240 command described in Section 9.1 (quit, help, reset, run, step, ustep, break, lsbrk, clear, save, load) and extends the state-query and state-set commands from Sections 9.2 and 9.3 to cover vector registers and the accumulator:

```
> v2?
v2: 0807060504030201

> v*?
v0: 0000000000000000 v1: 0807060504030201
v2: 0101010101010101 v3: 0000000000000000
v4: 0000000000000000 v5: 0000000000000000
v6: 0000000000000000 v7: 0000000000000000

> ACC?
ACC: 00000024
```

As with scalar registers, you can set a vector register or the accumulator directly by using an equals sign instead of a question mark:

```
> v3=0102030405060708
> ACC=00000000
```

v*? and *? both include the vector register file and ACC in their output, in addition to the scalar registers and microarchitectural registers already reported by sim240.

11.10. Notes

• Scalar registers are named r0 through r7.

• Vector registers are named v0 through v7.

• Vector arithmetic operates on eight 8-bit elements.

• Vector memory operations transfer four 16-bit words; ACCST transfers two 16-bit words.

• VMUL stores the low eight bits of each element product.

• VRELU and VDOT interpret each 8-bit element as signed two's-complement data.

• VDOT accumulates into ACC; use VACLR before a new independent dot product.

• ACCST stores the current value of ACC to memory and does not clear or modify ACC.

• The opcode values listed in this document come from the instruction-state encodings in constants.sv.

• Internal states such as VLD1 through VLD11, VST1 through VST10, and ACCST1 through ACCST6 are microarchitectural FSM states, not programmer-visible instructions.

12. Acknowledgments

The RISC240 instruction set architecture, the base scalar assembly instructions in Sections 3–10, and the original as240 assembler and sim240 simulator are the work of Bill Nace & Saugata Ghose. The RISC240-ML extension described in Section 11, including the vector register file, accumulator, VADD/VMUL/VRELU/VDOT/VACLR/VLD/VST/ACCST instructions, and the mlasm/mlsim tooling, was added by Leo Serodio.